

Restaurant Menu Pricing RAG

A retrieval system over publicly available restaurant menu data that produces cuisine-, tier-, and location-aware pricing comparisons, with confidence reported from the actual sample size.

Andrew McCombs · Independent Portfolio Project · Phoenix, AZ

Status. Independent portfolio project, not connected to any employer or commercial product. Built on publicly available restaurant menu data to demonstrate retrieval system design, statistical honesty under thin data, and privacy-by-construction handling of competitive information. Stores only derived facts (dish names, prices, embeddings), never copies of menus or images.

1. The problem

Restaurants set menu prices mostly by feel, with occasional competitor spot-checks. The useful question is narrow and local: for a casual-dining Mexican restaurant in this part of town, what does a carne asada taco actually go for?

Answering that well means clearing three bars that simple competitor averages skip.

Comparing like with like

A single average over a taqueria and a white-tablecloth restaurant is misleading because they are different products at different positions in the market.

Accounting for location

The same cuisine at the same tier commands a real premium in an affluent submarket and a discount in a value market. A metro-wide number hides this and would tell a Scottsdale operator to leave money on the table and a West Valley operator to price above what the market bears.

Being honest about confidence

A median computed from three restaurants is a guess. A pricing tool that presents it with the same authority as one computed from thirty is worse than no tool, because it invites bad decisions with false confidence.

2. The honest finding

The modeling is the easy part. A vector search plus a grounded language-model summary is reliable, and was working early.

What actually gates a trustworthy pricing answer is data: how many independent restaurants you have for a given cuisine, tier, and location, and how much usable public data exists in the first place.

The engineering that earned its keep was the acquisition pipeline (handling multiple menu formats from publicly available sources, with manual capture as a fallback for the harder cases) and the honesty layer (confidence derived from restaurant count, output anonymized by construction).

3. System architecture

Query path (retrieval-augmented generation)

- ▶ 1. The dish query is embedded with an OpenAI embedding model.
- ▶ 2. A pgvector cosine search retrieves semantically similar menu items across restaurants. This is the step that lets *carne asada tacos* match *asada street taco* and *grilled steak taco* across restaurants that word their menus differently. It is what makes this a RAG rather than a SQL report.
- ▶ 3. Retrieved comparables are filtered and aggregated deterministically in SQL and Python, by cuisine, tier, serving type, and zone.
- ▶ 4. A Claude model writes the recommendation grounded in those aggregated figures. The model explains the numbers, it does not invent them.

Ingestion (the harder half)

Publicly available menus appear in many formats. The pipeline tries the cheapest path first and escalates only when it has to.

- ▶ Static HTML and schema.org JSON-LD when present.
- ▶ JavaScript-rendered single-page sites via a headless browser.
- ▶ Menus embedded in third-party iframe widgets across several common hosting platforms.
- ▶ Image menus, read with a vision model.
- ▶ PDF menus, read with the model's native document support.
- ▶ **Manual capture.** For restaurants where automated extraction was unreliable or not cost-effective for a portfolio project (sites with aggressive anti-automation measures, off-site menu hosting, or unusual formats), menus were captured manually and run through the same structured-extraction pipeline. This was a deliberate scope decision: the engineering value of the project lives in the statistical and architectural layers, not in chasing the long tail of edge-case acquisition.

Extraction uses tool-use, so the model returns schema-validated structured data rather than free text that has to be parsed. Long menus are chunked and image menus are batched, so a single oversized request never silently drops items.

Technology stack

Layer	Choice
Language	Python 3.12
Store	Postgres + pgvector (Supabase)
Embeddings	OpenAI text-embedding-3-small (1536-dim)

Layer	Choice
Extraction	Claude Haiku (text and image) via tool-use
Synthesis and vision OCR	Claude Sonnet
Rendering	Headless browser for dynamic content
Discovery	Google Places API (New)
Interface	FastAPI, Typer CLI
Packaging and quality	uv, ruff

4. Methodology

Structured extraction

Every item is extracted with four classifications that the downstream logic depends on.

- ▶ **Serving type.** A la carte, plate, combo, or side.
- ▶ **Unit count for normalization.** 3 for a three-taco plate, so plate prices can be compared to a la carte on a per-item basis.
- ▶ **Menu type.** Regular or catering.
- ▶ **Price kept both as the raw string and a parsed number.** This matters because real menus say “Market Price”, “12/16/20”, and “\$8 ea”, and a numeric column alone would silently drop or mangle those.

Retrieval and aggregation

Comparables come from an exact pgvector nearest-neighbor search. No approximate index is needed at this corpus size. A similarity threshold drops loosely related items so a taco query does not pull in bone marrow.

The matched set is then partitioned and summarized in code, which keeps the statistics auditable and the language model's job limited to explanation.

5. The pricing model

Tiers are per cuisine, computed from the data

A single dollar boundary for value-vs-upscale is meaningless across cuisines. In the working corpus the median restaurant prices out very differently by cuisine.

Cuisine	Approx. median restaurant price
American	~\$19.00

Cuisine	Approx. median restaurant price
Italian	~\$16.50
Mexican	~\$13.60
Pizza	~\$13.25
Barbecue	~\$12.00

So \$15 is value for American and upscale for barbecue. Tier boundaries are therefore derived from each cuisine's own price distribution once there are enough restaurants to support it, which also means they track inflation automatically rather than drifting out of date.

The sample size is restaurants, not items

Items within one restaurant are correlated, because a restaurant prices its whole menu high or low together. Reporting “40 priced items” when they come from four restaurants overstates confidence badly.

The system counts distinct restaurants and labels every recommendation.

Confidence label	Distinct restaurants
Insufficient	Fewer than 5
Low	5 to 9
Moderate	10 to 19
High	20 or more

The recommendation opens by stating that level and the count behind it.

Serving types are separated and normalized

A three-taco plate and a single a la carte taco are different products. The system reports each serving type's prices separately and also normalizes plate prices to a per-item figure (price divided by item count), so a plate and a single item can be compared on the same basis instead of being blended into a misleading average.

Catering is held out

Catering menus are priced per tray or per person and would corrupt both dine-in price statistics and the tier calibration. They are tagged at extraction and excluded from regular pricing and from the tier boundaries.

A bug surfaced here and was fixed: an early run let a catering-only PDF inflate a restaurant's median and mislabel its tier, which is exactly the kind of contamination the separation is meant to prevent.

6. Geographic submarkets

The metro is divided into four coarse zones (Scottsdale and the premium northeast, central Phoenix, the East Valley, and the West Valley), assigned from each restaurant's ZIP.

Pricing reports the query's zone figure next to the metro figure, each with its own confidence. The zone figure is computed on the same tier-filtered comparable set, so it isolates the location effect rather than picking up a zone's tier mix.

An earlier design that used a single zone-vs-metro multiplier was rejected during review for exactly that confound: it would have conflated a genuine location premium with the fact that one zone happens to hold pricier restaurants.

The zones reproduce the intuition operators already have. The hard part is again density: splitting a modest corpus by cuisine, tier, and zone fragments it fast, so the suburban zones reach trustworthy sample sizes only with continued collection.

7. Privacy by construction

Output never names an individual restaurant. Identity is stripped from the data before it reaches the language model, so a specific restaurant cannot be named even by accident, rather than relying on a prompt instruction the model might ignore.

Output speaks only in category terms, for example:

▮ *Casual-dining Mexican restaurants in the premium zone charge around \$X for carne asada tacos.*

This mirrors how this kind of competitive data is handled responsibly: aggregate, never attributed.

8. Data collection and corpus

Restaurants are discovered through the Google Places API by cuisine and city, which returns names and websites. A resumable batch job runs each website through the ingestion pipeline, deduplicating on URL and name and logging every failure with its cause. Sites the automated pipeline could not handle were either captured manually for inclusion or dropped from the working set, depending on how representative they appeared.

The working corpus spans five cuisines across the metro, measured in priced restaurants (the real sample size).

Cuisine	Priced restaurants
Mexican	71
American	61
Italian	50

Cuisine	Priced restaurants
Pizza	46
Barbecue	31

At the metro level all five are in high-confidence territory. By submarket, Scottsdale, central Phoenix, and the East Valley reach moderate confidence for the major cuisines, which makes cross-zone comparisons defensible for those. The West Valley remains the thin zone and is the current collection priority.

9. Limitations and next steps

This project taught me more about the messiness of public data than about modeling, and the limitations are worth stating plainly.

- ▶ **Public data coverage is uneven.** Restaurant websites vary widely in format, accessibility, and maintenance. A meaningful share of attempts produce no usable menu, which is normal for working with publicly available data on the open web. Manual capture covers some of the gap, but it does not scale, and I deliberately chose not to over-invest in long-tail acquisition for a portfolio project.
- ▶ **Off-site menus.** Many restaurants no longer host their own menu at all. It lives on a delivery platform or social media, where the data is not available for use in this kind of analysis.
- ▶ **Density gates geographic confidence.** The metro and the three populated zones are solid. The suburban zone needs continued collection before its numbers are trustworthy.
- ▶ **Vision and PDF extraction are slow.** Accurate but the slow and expensive path, so the pipeline avoids them unless the cheaper paths fail.

Highest-priority next steps

- ▶ 1. Finish collection in the thin suburban zone.
- ▶ 2. Add a fixed per-site time budget so one slow page cannot stall a batch, a failure mode I encountered during development and addressed pragmatically.
- ▶ 3. Tighten the ZIP parser, which occasionally grabs a street number instead of the postal code.

10. Closing thought

The interesting result of this project is not a model score. It is the demonstration that a useful pricing answer is mostly a data-acquisition and data-honesty problem, and that the right engineering response is to invest in reaching the data, segmenting it correctly, and refusing to overclaim when it is thin.

The system is built to say “not enough data yet” clearly and often, because in this domain that is the most valuable thing it can tell an operator.